

ORIGINAL ARTICLE

Impact of $O(n)$ Symmetric Preprocessing on Inversion Reduction and Sorting Algorithm Efficiency: An Experimental Evaluation Across Adaptive and Non-Adaptive Methods

Received on Date Month Year; Accepted on Date Month Year

Abstract

Quadratic sorting algorithms remain relevant in resource-constrained systems but are sensitive to inversion counts. This study investigates the impact of inversions on sorting performance and their reduction through preprocessing. The objective is to assess the effectiveness of this technique for adaptive and non-adaptive algorithms. Five algorithms were compared across six input topologies, using vectors of up to 10^5 elements for quadratic methods and 10^6 for quasilinear methods with $O(n)$ preprocessing. The technique reduced inversions by 40–62% on non-aligned inputs at about 1.2 ns per element, with extreme reductions (99.99–100%) restricted to inputs geometrically aligned with the symmetric scan (Reversed, Zigzag). On non-aligned inputs, adaptive quadratic methods gained 13–42% (speedups up to $\times 2,073$ on aligned inputs), while the evaluated $O(n \log n)$ implementations showed mixed results for Merge Sort and systematic degradation for Quicksort.

Key words: Sorting algorithms, Adaptive sorting, Presorting, Benchmarking, Performance evaluation, Experimental methodology

1. Introduction

Sorting is a fundamental operation in computer science, essential in search systems, databases, and real-time applications [1, 2]. Although methods with $O(n \log n)$ asymptotic complexity, such as Merge Sort and Quicksort, are preferable for large data volumes, quadratic $O(n^2)$ algorithms such as Insertion Sort, Selection Sort, and Bubble Sort remain relevant in specific scenarios [3]. This persistence relates to structural simplicity, low memory consumption through in-place operation, and practical efficiency on small or partially ordered datasets [3, 4].

The practical efficiency of a sorting algorithm does not depend exclusively on its asymptotic complexity; it is also influenced by properties of the input sequence [5]. In this context, an inversion, defined as a pair satisfying $A[i] > A[j]$ for $i < j$, constitutes a quantitative measure of the disorder degree of an array [1, 6]. The inversion count characterizes different levels of presortedness, since sequences with more inversions exhibit greater disorder relative to the final order [6]. This characteristic can significantly influence execution cost, especially for algorithms whose operation count depends on input disorder [5].

Despite this relation between input disorder and execution cost, quadratic algorithms may exhibit limited adaptability to initial data conditions [5, 6]. In particular, inputs with many inversions can increase required movements during sorting, degrading performance of methods such as Insertion Sort [3]. In

this scenario, a preliminary step capable of reducing input disorder may decrease the subsequent work of the sorting algorithm. The research question is therefore: to what extent does a preprocessing step aimed at inversion reduction impact the practical performance of quadratic and quasilinear sorting algorithms?

Formally, let C_{original} denote the temporal cost of directly sorting the input. By introducing symmetric preprocessing with cost C_{pre} , the subsequent sorting algorithm operates on a modified vector, spending a reduced time C_{sort} . The objective of this work is to propose, implement, and evaluate a low-cost $O(n)$ symmetric preprocessing algorithm designed to reduce the inversion count of an input vector. We empirically verify whether the hybrid approach is more time-efficient than directly applying the same conventional algorithm, identifying the practical conditions under which $C_{\text{pre}} + C_{\text{sort}} < C_{\text{original}}$ holds. Comparison against the fastest available general sorter (rather than the same algorithm) and a formal closed-form theory of expected reduction are left as future work.

The relevance of this study lies in investigating a strategy capable of improving practical sorting behavior without modifying fundamental structure or introducing high additional cost. Reducing disorder before the main sort may decrease operations on unfavorable inputs, especially in methods sensitive to inversion counts [5, 7]. The study also contributes to adaptive-algorithm analysis by investigating conditions under

which preprocessing cost is compensated by reduced sorting work [6, 7].

The research adopts a quantitative, applied, and experimental approach [8, 9]. The study comprises formal modeling of the preprocessing algorithm and its implementation in Rust. Experimental evaluation used automated benchmarks with the Criterion library, employing synthetic vectors ranging from 1,000 to 1,000,000 elements with controlled inversion levels, covering worst cases, partial ordering, and adversarial structured topologies inspired by classical sorting-test patterns. Experiments compared execution time of the hybrid approach against directly applied sorting algorithms on the same inputs.

This article is structured as follows. Section 2 presents theoretical background. Section 3 describes methodology, formalization of the proposed preprocessing algorithm, evaluated sorting algorithms, and experimental procedures. Section 4 presents and discusses results. Finally, Section 5 presents conclusions.

2. Background and Related Work

This section presents the theoretical foundations: inversions as a disorder measure, adaptive-sorting principles, characteristics of evaluated algorithms including quadratic and quasilinear methods, preprocessing concepts and symmetric-preprocessing principles, and related work situating this contribution.

2.1. Inversions and Adaptive Sorting

Practical efficiency of traditional sorting algorithms often depends on initial element disposition [3, 5]. To formalize the disorder degree, the classical concept of *inversion* is used [1]. Given a vector a of n elements, an inversion is formally a pair (i, j) such that $i < j$ and $a[i] > a[j]$ [1, 2]. The total inversion count I measures input disorder and relates directly to effort required to sort via local operations [6].

The relation between inversion volume I and execution complexity gives rise to *adaptivity* [5]. An algorithm is adaptive when its time complexity improves markedly in the presence of partial presortedness [6]. A classical example is *Insertion Sort*, consuming $O(n + I)$ operations. On a strictly ordered vector ($I = 0$) it reaches linear $O(n)$ time, while in the worst case ($I = O(n^2)$) it degrades to quadratic time [1, 3].

2.2. Evaluated Algorithms

To evaluate symmetric-preprocessing impact across sorting paradigms, a representative set with distinct time complexity, adaptivity, and sensitivity to initial order was selected, covering both methods whose efficiency may be directly influenced by disorder and methods less dependent on it.

In the quadratic $O(n^2)$ worst-case group, *Bubble Sort*, *Insertion Sort*, and *Selection Sort* were evaluated [3]. *Insertion Sort* is adaptive in the inversion count ($O(n + I)$); *Bubble Sort* with early exit is adaptive in the number of passes—which is governed by the maximum left-displacement of any element rather than by the total I —while *Selection Sort* is non-adaptive and maintains fixed comparisons regardless of initial configuration [5]. This distinction matters for interpreting results: inversion elimination translates directly into fewer shifts for Insertion Sort, whereas Bubble Sort gains arise only when preprocessing collapses the maximum displacement (as in Zigzag and Reversed inputs).

In the quasilinear group, *Merge Sort* with $O(n \log n)$ time and *Quicksort* with $O(n \log n)$ average performance (though

$O(n^2)$ worst case) were evaluated [3]. Their inclusion verifies whether effects observed in quadratic algorithms remain relevant at lower asymptotic cost, extending analysis beyond inversion-count-related algorithms.

Thus the set covers three relevant dimensions: adaptive quadratic algorithms (*Bubble Sort* and *Insertion Sort*), a non-adaptive quadratic algorithm (*Selection Sort*), and quasilinear algorithms based on distinct strategies (*Merge Sort* and *Quicksort*). This composition verifies not only whether symmetric preprocessing reduces inversions, but to what extent this reduction converts into gains for algorithms with different dependence on initial order.

2.3. Symmetric Preprocessing Concept

Preprocessing, in algorithm design and analysis, refers to a preliminary transformation of input data before the main algorithm [3]. Its purpose is to alter instance disposition, making subsequent resolution simpler or more efficient. Conceptually, preprocessing acts analogously to prior organization of a work environment: although preparation requires initial cost, reduced main-processing effort compensates it.

For viability in sorting, preprocessing temporal cost must be strictly dominated by the succeeding sorting complexity [2, 3]. An ideal preprocessing should exhibit linear $O(n)$ time, ensuring overhead is asymptotically negligible relative to total sorting cost—whether $O(n \log n)$ average quasilinear or $O(n^2)$ worst-case quadratic [1].

In this research, preprocessing is termed *symmetric* because it applies a bidirectional mirrored scan from vector ends. This restructuring aims to reduce total inversions I by moving discrepant elements closer to final destinations in a single $O(n)$ pass. By providing a partially reordered structure, it seeks to mitigate comparisons and swaps in both adaptive and non-adaptive methods.

2.4. Related Work

Literature presents different approaches to characterize sequence sortedness and exploit it. Mannila [6] studied presortedness and established measures quantifying disorder forms. The author defined general properties for presortedness measures and introduced optimal sorting relative to a measure. Among measures is inversion count $Inv(X)$. The work presented optimal algorithms for different measures and showed *Insertion Sort* optimal for three natural measures, establishing foundations for adaptive algorithms [6].

Estivill-Castro and Wood [5] surveyed adaptive sorting, considering input disorder degree. The work presented main concepts, disorder measures, and established results on *adaptive sorting*, synthesizing theoretical and practical results including relations between disorder measures and algorithms exploiting presortedness. The survey records that several algorithms exhibit complexity dependent on both input size and disorder, and that certain adaptive algorithms perform competitively on random sequences with significant gains on nearly sorted inputs.

In an approach directly related to preliminary transformations, Hwang, Yang, and Yeh [7] investigated *presorting* algorithms from an average-case perspective, using inversion-count reduction as performance measure. The authors formulated quantification of preliminary-step effects via average inversion reduction, analyzing steps of known algorithms including *Shell-sort* and *Quicksort*, and derived expectation, variance, and generating functions of that reduction. Their framework could

in principle host a closed-form derivation of the expected reduction of symmetric preprocessing under uniform random input; such a derivation is left as future work. Empirically, the average reduction on random input was $\sim 45\%$ of the initial $\Theta(n^2)$ inversions (Table 3)—a large absolute $\Theta(n^2)$ reduction produced by a linear pass, whose precise expectation requires the generating-function machinery of [7].

Recent literature reinforces practical relevance of microarchitectural effects. Edelkamp and Weiss [10] showed branch mispredictions can dominate Quicksort cost and proposed *BlockQuicksort*, providing basis for degradation hypotheses for *Selection Sort* and Quicksort in this work. Auger et al. [11] analyzed *Timsort* worst-case complexity, showing exploitation of pre-existing runs remains an active sorting-engineering theme. LaMarca and Ladner [12] showed cache influence on sorting performance, also discussed by Kaligosi and Sanders [13] when quantifying branch-misprediction impact. These works position the present low-cost technique as a preprocessing-heuristic instance whose trade-off must be jointly evaluated at logical (inversions) and microarchitectural levels.

Differing from mapped approaches, this research evaluates a Symmetric Preprocessing algorithm acting on direct inversion mitigation in linear time, as one low-cost preprocessing instance. A systematic comparison against Shell-type passes and run-detection heuristics, and a formal gap analysis, remain future work.

3. Experimental Methodology

This work follows an applied, quantitative, experimental evaluation design [8, 9]: it controllably manipulates input topology and size (independent variables) and observes execution time and inversion counts (dependent variables) under a fixed seed, paired vector generation, and a standardized environment. From an evaluation-methodology standpoint, the study is framed as a cost–benefit assessment of a preprocessing pass: the measured total $C_{\text{pre}} + C_{\text{sort}}$ internalizes preprocessing overhead, the paired design controls input variability as a confounder, and correctness is checked outside the timed region. Three working hypotheses guide the analysis: (H1) adaptive quadratic algorithms benefit where absolute inversion reduction ΔI is large; (H2) the non-adaptive quadratic algorithm is neutral to inversion reduction; (H3) for quasilinear methods the overhead C_{pre} dominates unless structural disorder is extreme.

3.1. Experimental Configuration

Experiments ran in a controlled environment whose hardware, OS, and tool specifications appear in Table 1. To reduce variability, power plan was kept at high performance and background processes minimized. Code was compiled in *release* mode with maximum optimization (`opt-level = 3`). Input generation used a fixed seed, ensuring reproducibility and correspondence across experimental conditions.

To evaluate resilience under different entropy degrees, six structural input patterns were implemented with the exact generators in Table 2 (0-based indices, values in `i32`). For each topology-size combination, a single pool of 50 vectors is generated with `ChaCha8Rng` seeded by $42 \oplus n \oplus \text{type}$ and shared by all algorithms and both variants (pure and preprocessed), ensuring paired comparability. Deterministic generators (Reversed, Zigzag, Turtles) yield identical vectors across the 50 pool slots;

Table 1. Experimental environment and tools.

Component	Configuration / Specification
Operating system	Windows 11 Pro (build 10.0.26200)
Processor	AMD Ryzen 7 8700G
Memory	16 GB DDR5 5600MT/s
Language	Rust 1.96.0 (release profile)
Measurement tools	Criterion 0.5 and Project CLI Tool
Data generation	Seed 42 (<code>ChaCha8Rng</code>), paired inputs

Table 2. Input-topology generators (n = size, 0-based index i).

Topology	Exact Generator
Random	<code>shuffle</code> of $[0, n-1]$ with <code>ChaCha8Rng</code>
Turtles	$a[i] = i + n$ if $i < n/2$, else $a[i] = i \bmod 10$
Zigzag	$a[i] = i$ if i even, else $a[i] = n - i$
Almost Sorted	$[0, n-1]$ with $n/100$ uniform pair swaps (i, j) , $i \neq j$
Duplicates	$a[i] \sim \mathcal{U}\{0, 1, 2\}$ i.i.d.
Reversed	$a[i] = n - i$ (strictly decreasing)

stochastic generators (Random, Duplicates, Almost Sorted) advance the same RNG stream, so slot k differs across slots but is identical across algorithms and variants for fixed k .

Quasilinear algorithms (*Merge Sort* and *Quicksort*) were measured at $n \in \{1,000, 5,000, 10,000, 100,000, 1,000,000\}$. For quadratic algorithms (*Insertion Sort*, *Bubble Sort*, *Selection Sort*), measurements were restricted to $n \leq 100,000$: at $n = 10^6$ each execution would take minutes, precluding systematic collection of statistically significant samples. This restriction is documented and does not compromise comparison, as gain orders of magnitude already stabilize at $n = 100,000$.

Isolated preprocessing cost (C_{pre}) was additionally characterized at $n \in \{1,000, 5,000, 10,000, 20,000, 100,000, 1,000,000\}$ (Table 5 and Fig. 2) to evidence linearity; the $n=20,000$ point was collected exclusively for this cost curve via CLI tool (`seed 42`), under the same paired-generation protocol, and is not part of the main sorting-time comparison matrix.

3.2. Evaluated Algorithm Specifications

To ensure internal validity, timing stability, and reproducibility, all sorting algorithms were implemented in Rust (version 1.96.0), operating directly on mutable slices of 32-bit integers (`&mut i32`). Implementations avoided high-level external abstractions, standardizing memory handling and adopting idiomatic instruction-level optimizations.

3.2.1. Quadratic Algorithms

- **Bubble Sort:** Optimized adaptive version. Successive scans reduce upper bound each iteration ($n - 1 - i$). Adaptivity is ensured by a boolean flag (`swapped`) monitoring swaps per pass; if a scan completes with no element permutation, execution halts early, ensuring $O(n)$ on presorted vectors.
- **Insertion Sort:** Traditional direct-insertion by shifting. For each element of the unordered sub-slice, current value is stored in a temporary variable and strictly greater antecedents are shifted right. Search and shifting end once correct position is found or left bound ($j = 0$) is reached, minimizing write overhead versus successive-swap implementations.
- **Selection Sort:** Classical non-adaptive in-place form. At each outer iteration i , the algorithm scans unordered segment $[i+1, n-1]$ to identify minimum-element index. Swap

with position i executes conditionally only if minimum resides at a distinct index. Comparison cost remains strict $O(n^2)$, regardless of presortedness.

3.2.2. Quasilinear Algorithms

- **Merge Sort:** Top-down divide-and-conquer with stable ordering guarantee. To eliminate continuous dynamic-allocation overhead distorting timings, implementation allocates a single temporary auxiliary vector of size n at function entry, passed by mutable reference through recursive calls. Sub-slice midpoints use right bit-shifting ((end – start) $\gg$ 1), avoiding integer division. Merge copies current-slice data to auxiliary buffer and rebuilds ordered segment with three index pointers.
- **Quicksort:** Recursive implementation using three-way partitioning (*Dutch National Flag* / Dijkstra’s algorithm) with median-of-three pivot selection. Pivot is median among initial, central, and final subvector elements. Three-way partitioning rearranges into three contiguous regions: less than, equal to, and greater than pivot, maintaining **lt**, **i**, **gt** indices. This yields $O(n \log n)$ average time and robustness to many duplicate keys (no worst-case guarantee: median-of-three does not preclude $O(n^2)$ on adversarial inputs). Tail-call optimization iterates on the larger partition and recurses on the smaller.

In all implementations, preliminary boundary checks (*early returns*) ensure slices with $n < 2$ return immediately without function calls or unnecessary allocations.

3.3. Experimental Protocol and Inversion Counting

Empirical measurement used the *Criterion.rs* framework: warm-up cycles mitigate cache effects, and bootstrap resampling estimates 95% confidence intervals (**confidence_level=0.95**, **significance_level=0.05**). Each Criterion sample clones the next pool vector in rotation (**iter_batched**, per-iteration **clone** outside the timed region), so the 50-vector pool feeds all samples while cloning cost is never timed; only $C_{\text{total}} = C_{\text{pre}} + C_{\text{sort}}$ is timed in the preprocessed variant.

Measurement configuration was profile-adjusted (configured values; Criterion may extend effective measurement to reach statistical stability): 50 samples with 2s warm-up and 5s measurement window for most scenarios; for quadratic algorithms at $n = 100,000$ (iterations up to ≈ 2 s for Selection Sort), 30 samples with 1s warm-up and 5s window; and for quasilinear methods at $n = 10^6$, 50 samples with 2s warm-up and 10s window. Mean, median, and 95% CIs are computed per cell; tables report mean $\pm$ CI half-width.

Statistical criterion: a variation was considered relevant when 95% confidence intervals of the two conditions (pure vs. preprocessed) do not overlap. When CIs overlap, variation was treated as non-significant regardless of point percentage difference. This criterion is exploratory and conservative (the design is paired, but the analysis treats cells as independent); it was uniformly applied throughout Section 4 to qualify expressions such as “consistent gain,” “systematic degradation,” and “mixed results.” A paired bootstrap analysis of per-vector time ratios remains future work.

Beyond timing, correctness of each scenario was verified outside the timed region: for every (algorithm, topology, size), the sorted output is checked element-wise ($a[0] \leq a[1] \leq \dots$) and against **sort_unstable** in both pure and preprocessed variants. Inversion counts of each shared-pool vector, before and after

Algorithm 1 Symmetric Preprocessing Algorithm

Require: Vector A of size n
Ensure: Partially reorganized vector A

```

1:  $n \leftarrow \text{size of } A$ 
2: if  $n < 2$  then
3:   return
4: end if
5:  $\text{last} \leftarrow n - 1$ 
6:  $\text{mid} \leftarrow \lfloor n/2 \rfloor$ 
7: for  $i \leftarrow 0$  to  $\text{mid} - 1$  do
8:    $j \leftarrow \text{last} - i$ 
9:   if  $A[i] > A[j]$  then
10:     $\text{swap}(A[i], A[j])$ 
11:   end if
12:   if  $i + 1 < \text{mid}$  then
13:     if  $A[i] > A[i + 1]$  then
14:        $\text{swap}(A[i], A[i + 1])$ 
15:     end if
16:   end if
17:   if  $j - 1 > \text{mid}$  then
18:     if  $A[j] < A[j - 1]$  then
19:        $\text{swap}(A[j], A[j - 1])$ 
20:     end if
21:   end if
22: end for
```

preprocessing, are recorded in harness companion files, ensuring inversion tables derive from exactly the same timed vectors.

Exact inversion-count quantification uses a modified-MergeSort-based $O(n \log n)$ implementation [3]. During merging of two ordered subvectors L and R , when $R[j]$ is less than $L[i]$, exactly $|L| - i$ is added to the counter. This eliminates $O(n^2)$ direct-counting bottleneck, enabling evaluation of 10^5 – 10^6 -element vectors.

3.4. Symmetric Preprocessing Formalization

The proposed symmetric preprocessing operates as a deterministic single-pass scan over input vector A of size n . Its goal is restructuring via mirrored swaps and local adjustments before evaluated sorting algorithms. Procedural structure is formalized in Algorithm 1.

The algorithm iterates over the first half, scanning $i \in [0, \lfloor n/2 \rfloor - 1]$. Each iteration computes the symmetric index at the opposite end, $j = (n - 1) - i$. Loop execution comprises three structured conditionals: (1) **Extreme symmetric swap:** if $A[i] > A[j]$, direct swap places smaller element in left segment and larger in right; (2) **Left adjacent adjustment:** for $(i + 1) < \lfloor n/2 \rfloor$, if $A[i] > A[i + 1]$, swap adjacent elements; (3) **Right adjacent adjustment:** for $(j - 1) > \lfloor n/2 \rfloor$, if $A[j] < A[j - 1]$, swap adjacent elements.

Boundary conditions ensure adjacent swaps operate strictly within respective subdomains without crossing the midpoint: the left adjustment covers pairs up to $(\text{mid} - 2, \text{mid} - 1)$ and the right adjustment from $(\text{mid} + 1, \text{mid} + 2)$ onward, so neither side touches a midpoint-crossing adjacent pair and the procedure is left-right symmetric. Computationally, the algorithm executes exactly $\lfloor n/2 \rfloor$ iterations. Each step performs at most 3 comparisons and 3 swaps, yielding $T(n) = O(n)$ worst-case time. Since restructuring is in-place, auxiliary space is strictly $O(1)$. A useful monotonicity property is that swapping an inverted mirrored pair never increases I : it removes the (i, j) inversion and each intermediate k with $a_j < a_k < a_i$ contributes a net reduction,

Table 3. Inversion reduction by symmetric preprocessing on 10,000-element vectors (mean of 50 paired-pool vectors; same seed as Table 4).

Type	Inversions	Post-pre	Reduction
Random	24,960,120	13,681,366	45.19%
Turtles	30,613,750	18,133,750	40.77%
Zigzag	24,997,500	2,501	99.99%*
Almost Sorted	654,922	377,149	42.41%
Duplicates	16,633,993	6,300,346	62.12%
Reversed	49,995,000	0	100.00%**

* 99.99% (2,501 remaining); ** 100.00% exact (zero remaining). Rounding difference precludes equivalence interpretation.

so the pass is a descent step on I (a tight expected-reduction formula is future work).

3.5. Threats to Validity

Internal: single machine, OS (Windows 11), compiler (Rust 1.96.0, release, `opt-level=3`) and i32 data; CPU frequency scaling, SMT affinity, and Windows timer resolution were not controlled beyond high-performance power plan and background minimization. *Construct:* C_{pre} was characterized with the CLI tool averaged over topologies rather than per-topology within Criterion, so per-cell C_{pre} vs. ΔC_{sort} decomposition is unavailable; only i32 keys and one implementation per algorithm were tested. *External:* purely synthetic inputs, a single master seed (one 50-vector pool per cell), quadratic methods capped at $n \leq 10^5$, Duplicates fixed at $k=3$ values, and $n=10^6$ vectors ($\approx 4\text{MB}$) resident in the 8700G cache hierarchy—no claim extends beyond the cache-resident regime. *Statistical:* paired design analyzed with an unpaired, conservative non-overlap rule; paired bootstrap of per-vector ratios is pending. *Measurement:* no hardware counters (`perf_event_open` is Linux-specific); branch-predictor and cache attributions are hypotheses, not measurements. All experiments are reproducible via `cargo bench` and routines in the project repository [14].

4. Results and Discussion

This section analyzes experimental results, evaluating symmetric-preprocessing impact on data structure and algorithm execution time. Discussion follows an analytical structure from direct inversion-reduction impact and geometric-bias interpretation in the reversed scenario, through total sorting-time measurement in quadratic and quasilinear algorithms, to microarchitectural effects such as locality and CPU caches. The goal is consolidating scenarios with real gains and understanding physical and algorithmic limitations.

4.1. Inversion Reduction and Structural Bias

Fig. 1 illustrates inversion variation on 10,000-element vectors, with exact quantitative values and reduction percentages in Table 3. On non-aligned inputs (Random, Turtles, Almost Sorted, Duplicates) preprocessing removes 40–62% of inversions—a large absolute $\Theta(n^2)$ reduction. The near-total eliminations in *Reversed* (100%) and *Zigzag* (99.99%) occur precisely where disorder geometry coincides with the mirrored-scan geometry, and must be read as best-case alignment rather than typical behavior; the primary generalizable evidence is the Random and Almost Sorted columns.

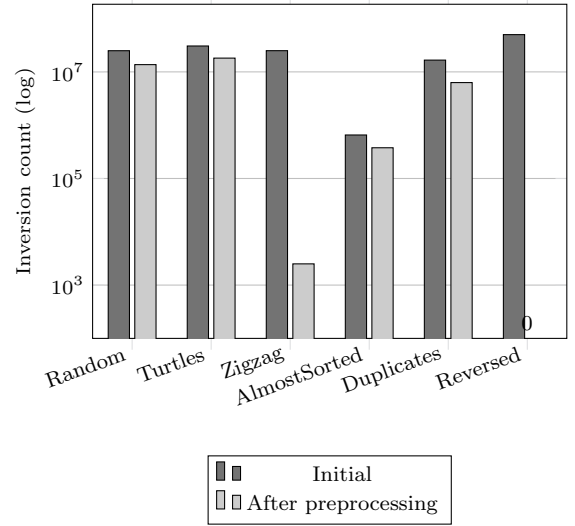


Figure 1. Inversion counts before and after symmetric preprocessing for 10,000-element vectors (log scale; the Reversed post value is exactly 0—annotated above the axis floor—and has no bar on the log scale).

Since Algorithm 1 directly swaps $a[i]$ and $a[n-1-i]$, a strictly reversed vector (where $a[i] > a[n-1-i]$ for all $i < n/2$) undergoes complete geometric reversal in $O(n)$ operations. Resolving Reversed stems from preprocessing-transformation geometry, an ideal case where disorder topology perfectly matches applied symmetry.

Behavior in *Zigzag* can also be interpreted via this structural property. Although its topology is not a global reversal like *Reversed*, systematic alternation between differently oriented blocks produces disorder relations partially captured by symmetric-position comparisons. Direct swapping between $a[i]$ and $a[n-1-i]$ thus simultaneously corrects many out-of-order relations, reducing inversions from 24,997,500 to only 2,501, approximately 100% (99.99%) reduction.

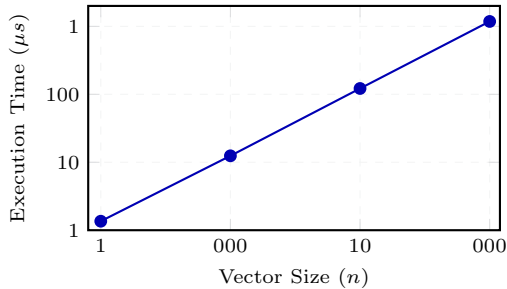
The result shows preprocessing efficacy is not restricted to complete reversal: it also manifests in topologies strongly aligned with transformation symmetry, even partially. This helps explain particularly high gains later observed in Insertion Sort and Bubble Sort for this input.

4.2. Execution-Time Impact and Preprocessing Cost

Gain is defined as $(T_{\text{pure}} - T_{\text{pre}})/T_{\text{pure}} \times 100\%$; equivalent speedups $T_{\text{pure}}/T_{\text{pre}}$ are given for saturated cells. Table 4 presents mean execution time and gain per algorithm on 10,000-element vectors, showing how preprocessing efficiency is conditioned by native complexity and adaptivity. On non-aligned inputs, *Insertion Sort* gained 28.6–59.3% (Random +42.2%) and *Bubble Sort* 13.3–27.5%. The largest benefits occur on aligned inputs: in *Zigzag*, preprocessing reduced *Insertion Sort* from $5,535.18\mu\text{s}$ to $16.44\mu\text{s}$ (+99.7%, $\times 337$ speedup) and *Bubble Sort* from $19,147.42\mu\text{s}$ to $17.67\mu\text{s}$ (+99.9%, $\times 1,084$ speedup); in *Reversed*, $\times 836$ and $\times 2,073$ speedups respectively (reported as +99.9% and +100.0% after rounding). In contrast, $O(n \log n)$ methods showed mixed results for *Merge Sort* (degradation up to 17.8% and gains up to 14.8% at $n=10,000$; gains up to 18.5% at $n=10^6$, Table 6). Since divide-and-conquer already ensures high native efficiency, preliminary-step cost tends to outweigh benefits in low-structure distributions. Variations were judged relevant by non-overlapping

Table 4. Mean time per execution (in μs , mean $\pm$ 95% CI half-width) and preprocessing gain for 10,000-element vectors. Source: Criterion, seed 42. Relevance: non-overlapping 95% CIs.

Algorithm	Type	Pure	With pre	Gain
Merge Sort	Random	429.35 $\pm$ 1.95	444.49 $\pm$ 5.94	-3.5%
Merge Sort	Turtles	125.17 $\pm$ 2.00	147.49 $\pm$ 4.30	-17.8%
Merge Sort	Zigzag	136.82 $\pm$ 1.30	116.63 $\pm$ 1.30	+14.8%
Merge Sort	Almost Sorted	135.04 $\pm$ 0.97	143.12 $\pm$ 1.31	-6.0%
Merge Sort	Duplicates	199.19 $\pm$ 1.70	201.32 $\pm$ 1.47	-1.1%
Merge Sort	Reversed	129.15 $\pm$ 2.41	112.21 $\pm$ 0.91	+13.1%
Quicksort	Random	367.27 $\pm$ 2.03	382.50 $\pm$ 1.25	-4.1%
Quicksort	Turtles	197.67 $\pm$ 2.55	207.13 $\pm$ 2.41	-4.8%
Quicksort	Zigzag	540.28 $\pm$ 9.78	562.13 $\pm$ 8.28	-4.0%
Quicksort	Almost Sorted	306.46 $\pm$ 2.16	315.72 $\pm$ 2.55	-3.0%
Quicksort	Duplicates	44.92 $\pm$ 0.43	59.00 $\pm$ 0.47	-31.3%
Quicksort	Reversed	471.18 $\pm$ 3.96	644.59 $\pm$ 5.75	-36.8%
Insertion Sort	Random	5506.86 $\pm$ 73.84	3181.76 $\pm$ 63.63	+42.2%
Insertion Sort	Turtles	6618.39 $\pm$ 79.22	4037.39 $\pm$ 76.08	+39.0%
Insertion Sort	Zigzag	5535.18 $\pm$ 57.97	16.44 $\pm$ 0.73	+99.7%
Insertion Sort	Almost Sorted	182.82 $\pm$ 3.93	130.58 $\pm$ 2.05	+28.6%
Insertion Sort	Duplicates	3662.06 $\pm$ 39.36	1488.72 $\pm$ 26.80	+59.3%
Insertion Sort	Reversed	10897.85 $\pm$ 160.40	13.04 $\pm$ 0.26	+99.9%
Bubble Sort	Random	23676.19 $\pm$ 117.13	20530.09 $\pm$ 203.74	+13.3%
Bubble Sort	Turtles	19428.15 $\pm$ 133.43	14094.37 $\pm$ 94.62	+27.5%
Bubble Sort	Zigzag	19147.42 $\pm$ 117.96	17.67 $\pm$ 0.43	+99.9%
Bubble Sort	Almost Sorted	16466.39 $\pm$ 83.51	13943.78 $\pm$ 141.98	+15.3%
Bubble Sort	Duplicates	20376.82 $\pm$ 100.35	17055.89 $\pm$ 97.02	+16.3%
Bubble Sort	Reversed	21307.46 $\pm$ 255.90	10.28 $\pm$ 0.40	+100.0%
Selection Sort	Random	21487.72 $\pm$ 137.81	21390.89 $\pm$ 106.85	+0.5%
Selection Sort	Turtles	21429.88 $\pm$ 171.35	20717.84 $\pm$ 122.11	+3.3%
Selection Sort	Zigzag	21382.51 $\pm$ 180.13	20975.92 $\pm$ 90.55	+1.9%
Selection Sort	Almost Sorted	21226.00 $\pm$ 116.05	21457.09 $\pm$ 279.79	-1.1%
Selection Sort	Duplicates	20626.01 $\pm$ 77.19	20788.64 $\pm$ 142.61	-0.8%
Selection Sort	Reversed	18262.67 $\pm$ 121.85	20515.49 $\pm$ 49.68	-12.3%

**Figure 2.** Mean preprocessing execution time vs. vector size (n), log-log scale evidencing $O(n)$ linear behavior. Means over six input types, CLI tool seed 42.

95% CIs (Section 3.3); non-significant cells (overlapping CIs) at $n=10,000$ are Merge/Duplicates (-1.1%) and Selection in Random, Almost Sorted, and Duplicates.

Practical viability rests on low, highly predictable symmetric-step cost. As shown in Table 5 and Fig. 2, preprocessing execution time is strictly linear $O(n)$ in vector size, with unit time between 1.18 and 1.36 ns per element, decaying monotonically with n and stabilizing at ≈ 1.18 ns for 10^6 elements, indicating no memory-locality bottlenecks at scale. The scan thus imposes negligible overhead—only $12.44 \mu\text{s}$ for 10,000 elements—amply compensated by millisecond savings in adaptive sorting.

Table 5. Mean symmetric-preprocessing cost vs. vector size (mean over six types; CLI tool, seed 42). Point $n=20,000$ included exclusively for linearity characterization.

Size	Pre (μs)	ns/element
1,000	1.36	1.36
5,000	6.25	1.25
10,000	12.44	1.24
20,000	24.56	1.23
100,000	121.89	1.22
1,000,000	1,181.84	1.18

4.3. Total Sorting Time and Per-Algorithm Analysis

Total-time evaluation reveals a dichotomy: adaptive quadratic algorithms gain wherever absolute inversion reduction ΔI is large (13.3–99.9% at $n=10,000$), because the modest linear scan (≈ 1.2 ns/element) is amortized by suppressed local work. Mechanistically, *Insertion Sort* time per inversion is strikingly regular— ≈ 0.22 ns/inversion on Random, Turtles, Zigzag, Reversed, and Duplicates at $n=10,000$ (e.g. $5,506.86 \mu\text{s}/24,960,120$)—so its gains track ΔI directly (Almost Sorted deviates at 0.28 ns/inversion, consistent with overhead dominance at small I). *Bubble Sort* time, by contrast, is nearly flat (16–24 ms) across inputs whose I spans two orders of magnitude: its pass count follows the maximum left-displacement, not I . Hence its modest Random gain (+13.3% despite -45% inversions) versus collapse to ≈ 1 pass on

aligned inputs ($\times 1,084$ – $2,073$ speedups). In contrast, $O(n \log n)$ methods respond qualitatively distinctly.

Merge Sort shows mixed results by non-overlapping-CI criterion. Gain concentrates in topologies where preprocessing eliminates structural disorder: *Reversed* (+13.1%) and *Zigzag* (+14.8%) at $n = 10,000$, with increasing gains to +18.5% at $n = 10^6$ (Table 6). In *Random*, *Turtles*, and *Almost Sorted*, however, the total degrades significantly. Part of these degradations is arithmetic: e.g. Merge/Random at $n=10,000$ slows by $+15.1 \mu s$ against a mean C_{pre} of $12.44 \mu s$, so overhead alone accounts for most of the gap; per-topology C_{pre} decomposition (same-harness measurement) is pending (Section 3.5).

Quicksort, implemented with three-way partitioning and median-of-three, shows systematic degradation under preprocessing. Loss is mild in most topologies (-3% to -5% at $n=10,000$ in *Random*, *Turtles*, *Zigzag*, *Almost Sorted*), but sharp in *Duplicates* (-31.3%) and *Reversed* (-36.8%), always with non-overlapping CIs. Again the overhead scale matters: Quicksort/Duplicates slows by $+14.08 \mu s$ at $n=10,000$ (mean C_{pre} $12.44 \mu s$) and $+1.31 ms$ at $n=10^6$ (mean C_{pre} $1.18 ms$), so most of the degradation is the preprocessing cost itself rather than a disturbed key distribution. The *Reversed* case additionally exposes an implementation-specific anomaly: the preprocessed (sorted) vector sorts *slower* ($591.80 ms$ at $n=10^6$) than the random one ($55.75 ms$), and scaling from $n=10^4$ to 10^6 is $\approx 884\times$ on *Reversed* ($\approx n^{1.45}$) versus $\approx 147\times$ on *Random* ($\approx n^{1.08}$). Median-of-three favors sorted inputs in theory, so no median-based explanation is offered; the partitioning behavior on ordered inputs in this implementation requires dedicated validation (comparison counts, reference quicksort), left as future work.¹

Thus for the $O(n \log n)$ class, the technique is not beneficial in the evaluated implementation, with point exceptions (Merge/Zigzag and Merge/Reversed at both scales; Quicksort/Almost Sorted $+11.1\%$ only at $n=10^6$, after -3.0% at $n=10,000$ and -9.3% at $n=10^5$). These results delimit preprocessing applicability, indicating inversion reduction alone does not guarantee gains for this class.

Selection Sort behavior is consistent with its non-adaptive nature: since comparison count is fixed ($n(n-1)/2$) regardless of initial configuration, inversion reduction yields no systematic gain. Variations span -1.1% to $+3.3\%$; by the stated non-overlap rule, *Random*, *Almost Sorted*, and *Duplicates* are non-significant (overlapping CIs), while *Turtles* ($+3.3\%$) and *Zigzag* ($+1.9\%$) are small but statistically significant deviations, and *Reversed* shows consistent $\approx 12\%$ degradation at $n = 10,000$ ($18,262.67 \pm 121.85 \mu s$ vs. $20,515.49 \pm 49.68 \mu s$; non-overlapping CIs), reproduced at $n = 100,000$ (-12.8% , $1,849.65 \pm 9.39 ms$ vs. $2,086.10 \pm 7.92 ms$). Note the framing: the pure *Reversed* cell ($18.26 ms$) is the fast outlier among otherwise 20.5 – $21.5 ms$ cells, so the “degradation” is relative to an anomalously fast baseline.

Since preprocessing transforms reversed input into an ordered sequence, minimum search per scan would find the smallest element at the first candidate; expected gain would be merely avoiding one swap per iteration, not justifying observed degradation. One hypothesis involves microarchitectural

factors—specifically CPU *branch predictor* behavior [10, 15]—but both always-taken and never-taken branch patterns are highly predictable, so this hypothesis is weak as stated; assembly inspection (conditional-move vs. branch) and per-iteration minimum-update counts are needed to test it. It remains a hypothesis, not a measurement (unavailable on Windows, Section 3.5). Importantly, variation magnitude is limited (at most $\sim 12\%$) with no systematic gain—consistent with the prediction that a non-adaptive algorithm does not exploit inversion reduction [5].

By simultaneously accessing opposite vector ends, symmetric preprocessing introduces additional control dependencies and increases instructions per element. On random instances, internal swap conditions (`if A[i] > A[j]`) become hard for the branch predictor [10, 13]. Consequently, where inversion reduction is modest, architectural penalties and extra instruction cost outweigh logical-restructuring benefits.

Fig. 3 synthesizes this dynamic: expressive exploitation in adaptive quadratic methods, mixed *Merge Sort* response, systematic three-way-*Quicksort* degradation, and neutral non-adaptive *Selection Sort* (except *Reversed*). Table 6 complements analysis for $n=10^6$ in quasilinear methods.

Values at $n=10^6$ replicate the $n=10,000$ pattern: Merge Sort gains restricted to *Zigzag* (+18.5%) and *Reversed* (+16.2%), systematic Quicksort degradation except *Almost Sorted* (+11.1%), confirming applicability delimitation to adaptive methods. At $n=10^6$ the only overlapping-CI cell is Merge/Random (-1.5%); all other quasilinear cells are significant, including Merge/Duplicates (+2.9%).

Fig. 4 extends the picture across collected sizes ($n=1,000$ – 10^6 , same Criterion means): Insertion/Random gains are stable (34.5–44.2%), Bubble/Random modest with an uptick at 10^5 (13.0–25.6%), Merge/Zigzag and Merge/Reversed grow monotonically (7.5–18.5% and 4.0–16.2%), Quicksort/Random stays near -4 to -7% , and Quicksort/Almost Sorted is non-monotonic (-7.6 , -8.4 , -3.0 , -9.3 , $+11.1\%$), flipping sign only at 10^6 —evidence that size trends do not uniformly stabilize and must be reported per topology.

5. Conclusion

This work presented and evaluated an $O(n)$ symmetric preprocessing for inversion-count reduction in sorting-algorithm input sequences. On non-aligned inputs it removed 40–62% of inversions at $\approx 1.2 ns/element$, yielding 13–42% gains in adaptive quadratic algorithms (Insertion Sort and Bubble Sort) at $n=10,000$; near-total inversion elimination and $\times 300$ – $2,000$ speedups occurred only on symmetry-aligned inputs (*Reversed*, *Zigzag*) and must not be generalized. Gains materialize when sorting-cost reduction exceeds preliminary-step cost, satisfying $C_{pre} + C_{sort} < C_{original}$. All comparisons are against the same algorithm without preprocessing; comparison against the fastest available sorter is future work.

For the evaluated non-adaptive quadratic algorithm, *Selection Sort* showed no systematic gain (-1.1% to $+3.3\%$, significant only in *Turtles*, *Zigzag*, and *Reversed* under the stated rule), except consistent $\sim 12\%$ degradation on fully reversed input (non-overlapping 95% CIs at $n=10,000$ and $n=100,000$), hypothesized—not established—to involve microarchitectural effects. This theoretically predicted neutrality via comparison-count invariance ($n(n-1)/2$) delimits that inversion reduction does not convert into gain for non-adaptive methods.

¹ Times and inversion counts are direct measurements (Tables 3 and 4); partitioning and branch-prediction attributions are explanatory hypotheses, not direct measurements (unavailable on Windows, Section 3.5).

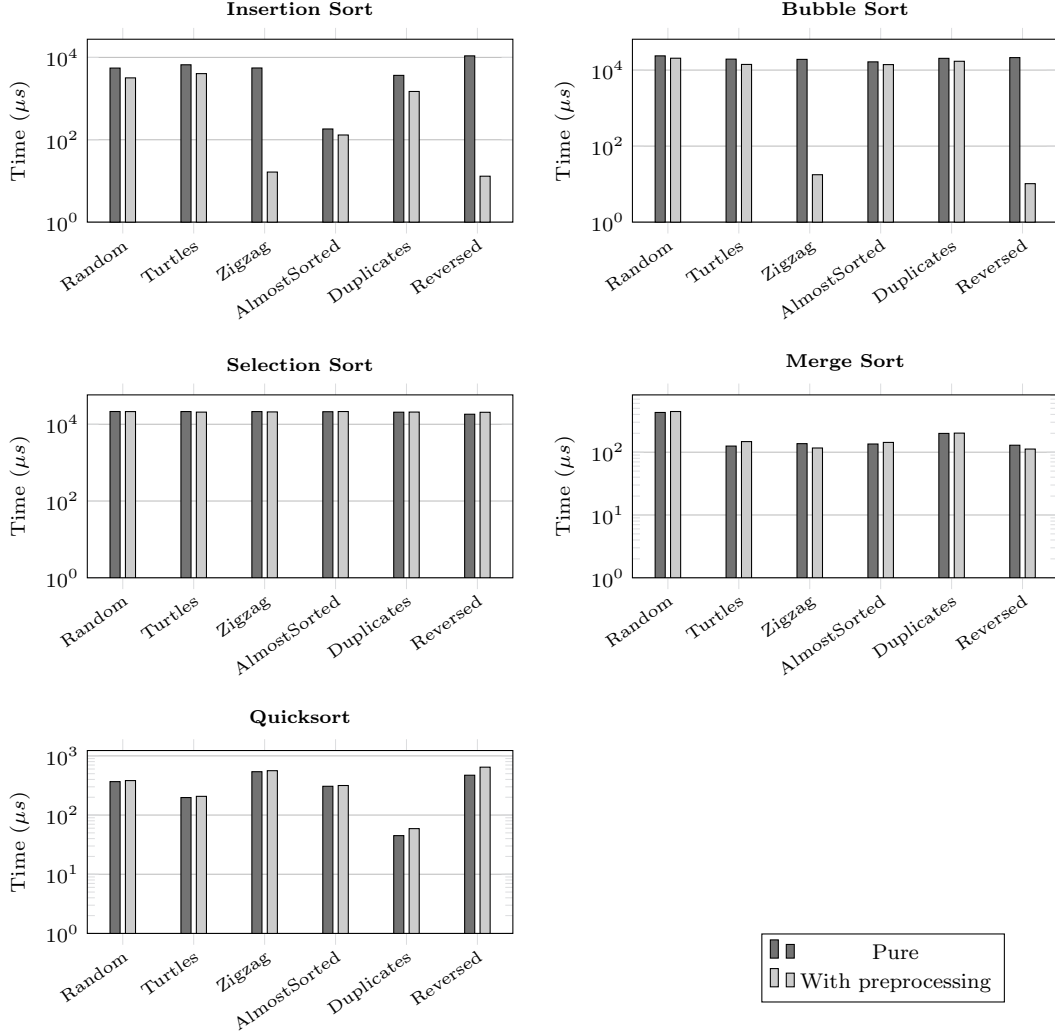


Figure 3. Temporal comparison between pure and symmetric-preprocessed sorting ($n = 10,000$), log scale.

Table 6. Mean time (ms) and preprocessing gain for 1,000,000-element vectors—quasilinear methods only (quadratic omitted for temporal infeasibility). Source: Criterion, 50 samples, 95% CI.

Algorithm	Type	Pure (ms)	With pre (ms)	Gain
Merge Sort	Random	66.14 $\pm$ 0.75	67.12 $\pm$ 0.76	-1.5%
Merge Sort	Turtles	17.92 $\pm$ 0.30	20.03 $\pm$ 0.51	-11.8%
Merge Sort	Zigzag	19.39 $\pm$ 0.39	15.81 $\pm$ 0.26	+18.5%
Merge Sort	Almost Sorted	20.39 $\pm$ 0.39	21.34 $\pm$ 0.39	-4.7%
Merge Sort	Duplicates	25.85 $\pm$ 0.42	25.09 $\pm$ 0.26	+2.9%
Merge Sort	Reversed	18.46 $\pm$ 0.47	15.46 $\pm$ 0.19	+16.2%
Quicksort	Random	53.83 $\pm$ 0.18	55.75 $\pm$ 0.26	-3.6%
Quicksort	Turtles	151.34 $\pm$ 0.86	157.07 $\pm$ 3.10	-3.8%
Quicksort	Zigzag	458.73 $\pm$ 2.59	506.44 $\pm$ 3.06	-10.4%
Quicksort	Almost Sorted	42.61 $\pm$ 0.27	37.88 $\pm$ 0.27	+11.1%
Quicksort	Duplicates	4.39 $\pm$ 0.02	5.70 $\pm$ 0.03	-29.9%
Quicksort	Reversed	416.59 $\pm$ 2.02	591.80 $\pm$ 4.62	-42.1%

In evaluated $O(n \log n)$ algorithms, the method revealed important limitations. With the used Quicksort implementation (three-way partitioning and median-of-three), preprocessing caused systematic degradation—about 3%–5% at $n=10,000$ in most topologies (-10.4% for Zigzag at $n=10^6$) and $\approx 30\%$ – 42%

in *Duplicates* and *Reversed* (non-overlapping CIs)—largely on the scale of preprocessing overhead itself, plus an unexplained ordered-input anomaly in this implementation that requires dedicated validation.

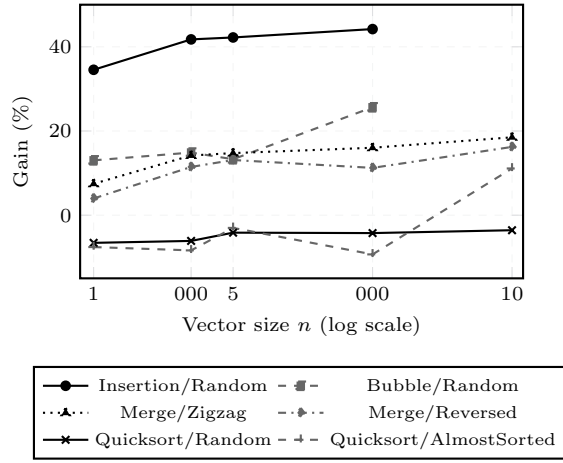


Figure 4. Preprocessing gain vs. input size for selected algorithm-topology pairs (same Criterion means as Tables 4–6; quadratic methods capped at $n=10^6$).

In Merge Sort effects were mixed, with gains restricted to topologies most structurally affected by symmetric scanning (Zigzag +14.8% at $n=10,000$ and +18.5% at $n=10^6$; Reversed +13.1% and +16.2%) and degradation elsewhere. Practical viability must therefore be strictly conditioned on adaptive quadratic algorithms, not extending to quasilinear methods generally and particularly not benefiting non-adaptive algorithms like Selection Sort.

As future work, we recommend (i) closed-form theoretical derivation of expected inversion reduction under random input in Hwang et al.’s framework [7], including the swap-monotonicity lemma sketched in Section 3.4; (ii) competitive baselines (`sort_unstable`, run-detection with $O(n)$ reversal) and non-aligned topologies including small n (16–512) and real data; (iii) same-harness per-topology C_{pre} decomposition, paired bootstrap inference, multi-seed pools, and Quicksort validation against a reference implementation; (iv) Linux-environment tests with `perf_event` infrastructure, enabling direct *cache-miss* and *branch-misprediction* measurement; and (v) alternative partitionings and adaptive techniques such as *BlockQuicksort* [10] and *Timsort* [11] as comparative baselines.

References

1. D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*. Reading, MA: Addison-Wesley, 1973.
2. R. Sedgewick and K. D. Wayne, *Algorithms*, 4th ed. Upper Saddle River: Addison-Wesley, 2011.
3. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.
4. N. Ziviani, *Projeto De Algoritmos Com Implementações Em Java E C++*. Cengage Learning, Aug. 2021.
5. V. Estivill-Castro and D. Wood, “A survey of adaptive sorting algorithms,” *ACM Computing Surveys*, vol. 24, no. 4, pp. 441–476, Dec. 1992, doi: <https://doi.org/10.1145/146370.146381>. [Online]. Available: <https://dl.acm.org/doi/10.1145/146370.146381>
6. H. Mannila, “Measures of presortedness and optimal sorting algorithms,” *IEEE Transactions on Computers*, vol. C-34, no. 4, pp. 318–325, 1985, preliminary version

- in Proc. 11th ICALP, LNCS 172, pp. 324–336, 1984. doi: <https://doi.org/10.1109/TC.1985.5009382>.
7. H.-K. Hwang, B.-Y. Yang, and Y.-N. Yeh, “Presorting algorithms: An average-case point of view,” *Theoretical Computer Science*, vol. 242, no. 1-2, pp. 29–40, Jul. 2000, doi: [https://doi.org/10.1016/S0304-3975\(98\)00181-9](https://doi.org/10.1016/S0304-3975(98)00181-9). [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0304397598001819>
8. A. C. Gil, *Métodos e técnicas de pesquisa social*. Editora Atlas Ltda, Feb. 2019.
9. R. H. Sampieri, C. F. Collado, and M. d. P. B. Lucio, *Metodologia de pesquisa*. Penso, Nov. 2021.
10. S. Edelkamp and A. Weiß, “Blockquicksort: Avoiding branch mispredictions in quicksort,” in *Proceedings of the 24th Annual European Symposium on Algorithms (ESA 2016)*, ser. LIPIcs, vol. 57, 2016, pp. 38:1–38:16, doi: <https://doi.org/10.4230/LIPIcs.ESA.2016.38>. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2016.38>
11. N. Auger, V. Jugé, C. Nicaud, and C. Pivoteau, “On the worst-case complexity of timsort,” in *Proceedings of the 26th Annual European Symposium on Algorithms (ESA 2018)*, ser. LIPIcs, vol. 112, 2018, pp. 4:1–4:13, doi: <https://doi.org/10.4230/LIPIcs.ESA.2018.4>. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ESA.2018.4>
12. A. LaMarca and R. E. Ladner, “The influence of caches on the performance of sorting,” *Journal of Algorithms*, vol. 31, no. 1, pp. 66–104, Apr. 1999, doi: <https://doi.org/10.1006/jagm.1998.0985>. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0196677498909853>
13. K. Kaligosi and P. Sanders, “How branch mispredictions affect quicksort,” in *Proceedings of the 14th Annual European Symposium on Algorithms (ESA 2006)*, ser. LNCS, vol. 4168. Springer, 2006, pp. 780–791, doi: https://doi.org/10.1007/11841036_69.
14. “Presorting algorithm – experimental framework and data,” Anonymous repository for double-blind review, 2026, source code and experimental data available upon acceptance.
15. J. L. Hennessy and D. A. Patterson, *Computer architecture: a quantitative approach*, sixth edition ed. Cambridge, Massachusetts: Elsevier, 2019.